

Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»
(МИ ВлГУ)

Факультет информационных технологий
Кафедра информационных систем

***КУРСОВАЯ
РАБОТА***

по курсу Прикладная разработка на Java
на тему: Разработка веб-приложения для учета расходов семьи

Руководитель

К. Т. Н., доц. каф. ИС
(уч. степень, звание)

Метелкин А.С.
(фамилия, инициалы)

(подпись) (дата)

Студент ИС-123
(группа)

Коротков М.И.
(фамилия, инициалы)

(подпись) (дата)

(оценка)

Члены комиссии

(подпись) (Ф.И.О.)

(подпись) (Ф.И.О.)

Данная курсовая работа посвящена разработке веб-приложения на языке Java для управления семейным бюджетом. Система обеспечивает учет доходов и расходов, планирование бюджета и формирование отчетности с использованием реляционной базы данных и трех механизмов взаимодействия: веб-интерфейса, REST API и модуля экспорта данных.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1.АНАЛИЗ ТЕХНИЧЕСКОГО ЗАДАНИЯ.	6
1.1.ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.2. ОПИСАНИЕ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ	7
1.3. МОДЕЛЬ ДОСТУПА ПОЛЬЗОВАТЕЛЕЙ	9
1.4. БЕЗОПАСНОСТЬ ДАННЫХ.....	10
1.5. ИНСТРУМЕНТАЛЬНЫЕ СРЕДСТВА РАЗРАБОТКИ	11
1.6 АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ	13
2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ	18
2.1. СХЕМА ВЗАИМОДЕЙСТВИЯ ИНТЕРФЕЙСНЫХ МОДУЛЕЙ	18
2.2. МАТРИЦА ДОСТУПА	21
3. РАЗРАБОТКА ПРИЛОЖЕНИЯ	22
3.1. АРХИТЕКТУРА ПРИЛОЖЕНИЯ.....	22
3.2. РЕАЛИЗАЦИЯ ТРЕХ ДВИЖКОВ ИНТЕРФЕЙСА	25
3.3. СТРУКТУРА ПРОЕКТА.....	30
3.4. ОСНОВНЫЕ ОКНА ПРИЛОЖЕНИЯ	31
4.ТЕСТИРОВАНИЕ.....	35
ЗАКЛЮЧЕНИЕ	39
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ.	41

					МИВУ 09.03.02 - 00.008 ПЗ						
Изм.	Лист	№ докум.	Подп.	Дата							
Разраб.	Коротков М.И.				Разработка веб-приложения для учета расходов семьи	Лит.			Лист	Листов	
Пров.	Метелкин А.С..					у			4	41	
						МИ ВлГУ ИС-123					
Н. контр.											
Утв.											

Введение

Современные семьи сталкиваются с необходимостью эффективного управления личными финансами в условиях экономической нестабильности. Согласно исследованиям, 68% семей не ведут систематический учет доходов и расходов, что приводит к нерациональному использованию бюджета и невозможности достижения долгосрочных финансовых целей.

- Разрабатываемое веб-приложение решает следующие задачи:
- Автоматизация учета доходов и расходов с детализацией по категориям
- Многопользовательский режим работы с разграничением прав доступа
- Планирование бюджета с визуальным контролем исполнения
- Управление целями накопления с отслеживанием прогресса
- Формирование аналитической отчетности в различных форматах
- Обеспечение безопасности данных и конфиденциальности информации

Приложение предназначено для использования в домашних хозяйствах и представляет собой полнофункциональную систему управления семейными финансами с возможностью работы с различных устройств (десктоп, планшет, мобильный телефон).

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		5

1. Анализ технического задания.

1.1. Описание предметной области

Предметной областью разрабатываемого программного комплекса выступает сфера автоматизации процессов управления личными и семейными финансовыми потоками. В современных экономических реалиях эффективное распределение денежных средств требует не только фиксирования транзакций, но и глубокого аналитического осмысления структуры расходов, планирования будущих трат и контроля за достижением поставленных финансовых ориентиров. Разрабатываемая информационная система охватывает полный цикл взаимодействия пользователей с финансовыми данными, начиная от первичного ввода операций и заканчивая формированием сводной аналитической отчетности.

Функциональная архитектура приложения построена вокруг нескольких ключевых подсистем, обеспечивающих комплексное решение поставленных задач. Модуль аутентификации и авторизации отвечает за идентификацию субъектов системы, разграничение прав доступа и ведение журнала аудита событий. Финансовый блок реализует механизмы учета поступающих доходов и фиксируемых расходов с обязательной привязкой к тематическим категориям и конкретным членом семьи, что позволяет формировать детальную картину денежных потоков. Планировочный модуль предоставляет инструменты для установки месячных лимитов трат, сопровождая процесс визуальным контролем исполнения бюджета посредством динамических индикаторов, а также поддерживает функционал управления целями накопления с отслеживанием прогресса их достижения.

Аналитическая составляющая системы базируется на интерактивной визуализации данных, позволяющей пользователям исследовать структуру расходов через графические представления и применять гибкие фильтры по временным периодам, категориям и участникам. Заключительным элементом функционала выступает модуль отчетности, автоматизирующий процесс

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подп.	Дата		

генерации детализированных документов в табличном формате Excel с разбивкой по смысловым блокам, а также обеспечивающий возможность резервного копирования массива данных в структурированный формат JSON для целей архивирования и миграции.

1.2. Описание вариантов использования

Взаимодействие конечных пользователей с разрабатываемой системой регламентируется четкими сценариями использования, которые дифференцируются в зависимости от назначенной роли субъекта. Для пользователя, обладающего статусом Главы семьи (OWNER), предусмотрен расширенный перечень возможностей, включающий не только стандартные операции по ведению учета, но и административные функции, такие как управление учетными записями участников, настройка общих финансовых лимитов и формирование комплексной аналитической отчетности. Участники семьи (MEMBER) взаимодействуют с системой в рамках ограниченного контура прав, фокусируясь на внесении персональных транзакций, мониторинге общей статистики домохозяйства и управлении индивидуальными финансовыми целями.

Детализация ключевых сценариев использования выглядит следующим образом. Процесс аутентификации пользователя инициируется путем обращения к стартовой странице входа в приложение, где субъект вводит учетные данные в соответствующие поля интерфейса. Запрос передается на серверный компонент системы, который осуществляет верификацию введенного пароля путем сравнения его криптографического хэша, сформированного алгоритмом BCrypt, с эталонным значением, хранящимся в поле PASSWORD_HASH таблицы USERS. В случае успешного сопоставления данных система определяет уровень привилегий пользователя на основании значения поля ROLE и динамически загружает интерфейс, адаптированный под назначенную роль, либо выводит уведомление об ошибке валидации.

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подп.	Дата		

Сценарий внесения расходной операции участником семьи начинается с активации элемента интерфейса, предназначенного для создания новой транзакции. Система инициирует отображение модальной формы ввода, в которой пользователь указывает денежную сумму, выбирает соответствующую категорию трат из предоставленного перечня и при необходимости дополняет запись текстовым комментарием. Перед фиксацией данных выполняется многоуровневая проверка корректности введенных параметров, гарантирующая положительное значение суммы и обязательное указание категории. После подтверждения операции система формирует новую запись в таблице TRANSACTIONS, связывает её с идентификатором текущего пользователя, фиксирует временную метку выполнения и инициирует пересчет агрегированных показателей бюджета семьи. Успешное завершение процесса сопровождается выводом информационного уведомления и мгновенным обновлением аналитических виджетов на главной панели.

Формирование финансового отчета в табличном формате Excel осуществляется пользователем с расширенными полномочиями через специализированный раздел интерфейса. После выбора интересующего временного интервала и применения опциональных фильтров по категориям или участникам, система инициирует процесс агрегации данных путем выполнения структурированных SQL-запросов к основным таблицам хранения транзакций, доходов и справочной информации. Полученный массив данных проходит этап логической группировки и распределяется по соответствующим листам рабочей книги, после чего специализированная библиотека Apache POI осуществляет генерацию бинарного файла формата .xlsx, который автоматически передается в поток загрузки на устройство пользователя.

Управление механизмами накопления средств предполагает переход пользователя в соответствующий раздел системы, где отображается перечень активных финансовых целей с визуальной индикацией степени их заполнения. При создании новой цели субъект указывает её наименование, желаемый итоговый объем средств и определяет тип принадлежности (семейный или

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подп.	Дата		

персональный), после чего система регистрирует соответствующую запись в таблице SAVINGS_GOALS. В процессе пополнения цели система актуализирует поле текущего накопленного объема, производит автоматический расчет процентного соотношения выполненной части к целевому показателю и обновляет состояние визуальных индикаторов прогресса в пользовательском интерфейсе.

1.3. Модель доступа пользователей

Управление правами доступа в разрабатываемой системе реализовано на базе ролевой модели (RBAC), которая обеспечивает строгое разграничение функциональных возможностей в зависимости от статуса пользователя в структуре семьи. Права и ограничения определяются значением атрибута ROLE в сущности USERS, что позволяет гибко настраивать уровень взаимодействия каждого субъекта с данными системы.

Пользователи, отнесенные к категории Членов семьи (MEMBER), наделены базовым набором привилегий, достаточным для ведения личного финансового учета в рамках общего домохозяйства. Данный контур прав включает возможность создания записей о личных расходах и доходах, просмотр агрегированной информации о всех транзакциях семьи, мониторинг текущих финансовых остатков и статистических показателей, а также управление персональными целями накопления. При этом архитектура системы намеренно ограничивает доступ участников к административным функциям, исключая возможность влияния на настройки общих бюджетных лимитов или управления учетными записями других пользователей.

Роль Главы семьи (OWNER) подразумевает наследование всех базовых возможностей участника, дополненных расширенными административными и аналитическими полномочиями. Данный статус предоставляет полный контроль над управлением как семейными, так и персональными целями накопления всех участников домохозяйства, а также открывает доступ к детализированной

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						9
Изм.	Лист	№ докум.	Подп.	Дата		

аналитике, разбитой по каждому члену семьи. Ключевым отличием данной роли является наличие исключительных прав на установку и модификацию месячных лимитов бюджета через таблицу BUDGET_PLANS, управление справочником категорий расходов и формирование комплексных отчетных документов с возможностью экспорта. Кроме того, пользователь с данным статусом наделен полномочиями по администрированию профиля семьи в таблице FAMILIES, что обеспечивает централизованное управление структурой домохозяйства и его финансовыми параметрами.

1.4. Безопасность данных

Обеспечение информационной безопасности разрабатываемого веб-приложения реализуется посредством комплекса взаимосвязанных механизмов, направленных на защиту конфиденциальности данных, предотвращение несанкционированного доступа и сохранение целостности хранимой информации. Фундаментальным элементом подсистемы аутентификации выступает алгоритм хеширования паролей BCrypt с параметром сложности, установленным на уровне десяти раундов шифрования. Данный подход гарантирует, что исходные значения паролей пользователей никогда не сохраняются в базе данных в открытом виде, а хранятся исключительно в форме криптографических хэшей в поле PASSWORD_HASH таблицы USERS, что минимизирует риски компрометации учетных записей в случае утечки данных.

Разграничение функциональных возможностей осуществляется через строгую проверку ролевых атрибутов на уровне контроллеров и сервисных компонентов, где применяются аннотации механизма Spring Security для валидации прав доступа к защищенным эндпоинтам. Управление сессиями пользователей построено на базе протокола HTTP Session с настроенным автоматическим завершением неактивных сеансов по истечении тридцатиминутного интервала, что предотвращает возможность использования устаревших или заброшенных соединений. При инициировании процедуры

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						10
Изм.	Лист	№ докум.	Подп.	Дата		

выхода из системы происходит принудительное уничтожение сессионных данных и очистка связанных cookie-файлов.

Для защиты от распространенных веб-уязвимостей, таких как межсайтовая подделка запросов (CSRF), в архитектуре приложения интегрирован механизм генерации и верификации уникальных токенов для всех запросов, модифицирующих состояние системы. Целостность реляционных данных обеспечивается на уровне схемы базы данных посредством внедрения внешних ключей (FOREIGN KEY), которые предотвращают возникновение орфанных записей и нарушенных ссылочных связей между сущностями. Дополнительно, на уровне бизнес-логики реализуется строгая валидация входных параметров, проверяющая корректность денежных сумм, допустимость временных диапазонов и заполненность обязательных полей, что исключает попадание некорректных данных в хранилище. Все значимые события, включая процедуры входа, финансовые операции и экспортные действия, фиксируются в журнале аудита AUDIT_LOGS с привязкой к идентификатору пользователя, временной метке и описанию выполненного действия, что обеспечивает прозрачность и возможность ретроспективного анализа активности в системе.

1.5. Инструментальные средства разработки

Выбор технологического стека для реализации разрабатываемого программного комплекса обусловлен требованиями к высокой производительности, масштабируемости, безопасности и соответствию современным стандартам enterprise-разработки. В качестве основного языка программирования задействована Java версии 21 (LTS), которая предоставляет долгосрочную поддержку, оптимизированную JIT-компиляцию для обеспечения высокой скорости выполнения вычислений, а также строгую статическую типизацию, критически важную для финансовых приложений, оперирующих точными денежными значениями. Богатая экосистема библиотек и фреймворков

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		11

Java существенно ускоряет процесс разработки и интеграции сторонних компонентов.

Серверная архитектура приложения построена на базе фреймворка Spring Boot версии 3.2.4, который предоставляет механизмы внедрения зависимостей (DI) и инверсии управления (IoC), позволяя создавать слабосвязанные, легко тестируемые и сопровождаемые программные модули. Автоматическая конфигурация и встроенный сервер приложений Tomcat существенно упрощают процесс развертывания и снижают накладные расходы на настройку окружения. Для абстрагирования логики взаимодействия с реляционным хранилищем используется спецификация Spring Data JPA совместно с реализацией Hibernate 6.4, что обеспечивает поддержку методов запросов по именам, автоматическую генерацию SQL-инструкций, управление транзакциями и многоуровневое кэширование данных.

В качестве системы управления базами данных в проекте задействована СУБД РЕД База данных, представляющая собой надежное отечественное решение, совместимое со стандартами PostgreSQL, что гарантирует поддержку транзакционности, ссылочной целостности и возможность миграции на промышленные кластерные решения без модификации кода приложения. Клиентская часть системы реализуется посредством шаблонизатора Thymeleaf 3.1, обеспечивающего серверный рендеринг HTML-страниц с естественным синтаксисом, и адаптивной CSS-библиотеки Bootstrap 5.3, предоставляющей готовые компоненты для построения отзывчивого интерфейса. Визуализация аналитических данных осуществляется с использованием библиотеки ApexCharts, поддерживающей интерактивные SVG-графики.

Подсистема информационной безопасности базируется на модуле Spring Security 6.2, который предоставляет комплексные инструменты для аутентификации, авторизации и защиты от типовых веб-атак. Генерация табличных отчетов делегирована библиотеке Apache POI версии 5.2.5, специализирующейся на работе с форматами Microsoft Office. Управление зависимостями, жизненным циклом сборки и плагинами автоматизировано

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		12

посредством инструмента Apache Maven 3.9. Разработка и отладка кода осуществляются в интегрированных средах Apache NetBeans или IntelliJ IDEA, а контроль версий и совместная работа над исходным кодом организованы через распределенную систему Git.

1.6 Анализ существующих решений

На современном рынке программного обеспечения представлен широкий спектр информационных систем, предназначенных для автоматизации процессов управления личными и семейными финансами. Для обоснования целесообразности разработки собственного веб-приложения был проведен сравнительный анализ наиболее распространенных аналогов, выявлены их сильные и слабые стороны, а также определены ключевые отличия разрабатываемой системы.

К числу основных представителей данной предметной области относятся следующие программные продукты:

Дзен-мани (Zenmoney) представляет собой облачный сервис, ориентированный на мобильные платформы. Система предоставляет функционал автоматической синхронизации с банковскими счетами, категоризации транзакций и планирования бюджета. Однако архитектура решения предполагает хранение данных на сторонних серверах, что может вызывать вопросы у пользователей, требующих полного контроля над конфиденциальной финансовой информацией. Кроме того, система является проприетарной с закрытым исходным кодом.

CoinKeeper — это мобильное приложение, использующее визуальный интерфейс в виде «монеток» для упрощения процесса учета расходов. Продукт ориентирован на индивидуальных пользователей и отличается минималистичным дизайном. В то же время, функциональные возможности системы ограничены базовым учетом операций без поддержки сложных

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		13

аналитических инструментов, многопользовательского режима или детального разграничения прав доступа.

YNAB (You Need A Budget) — платная веб-платформа, базирующаяся на собственной методологии бюджетирования. Система делает акцент на дисциплине расходов и долгосрочном планировании. Несмотря на наличие мощного аналитического аппарата, продукт распространяется по подписной модели, что создает финансовые барьеры для массового использования. Кроме того, как и в случае с Дзен-мани, данные хранятся в облачной инфраструктуре разработчика.

HomeBank — бесплатное десктопное приложение с открытым исходным кодом. Программа предоставляет базовый функционал для учета доходов и расходов, поддерживает импорт данных из банковских выписок и формирование простых отчетов. Основным недостатком данного решения является отсутствие веб-интерфейса, что препятствует использованию системы с мобильных устройств или планшетов, а также невозможность одновременной работы нескольких членов семьи с единой базой данных в реальном времени.

Monefy — минималистичное мобильное приложение, предназначенное для быстрого фиксирования транзакций. Интерфейс оптимизирован для оперативного ввода данных, однако аналитические возможности и функции совместного использования бюджета в системе практически отсутствуют.

1С:Предприятие (специализированные конфигурации) — комплексная корпоративная система учета. Хотя платформа обладает широчайшим функционалом, её внедрение требует значительных финансовых затрат на лицензирование, а также наличия квалифицированных специалистов для настройки и сопровождения. Для задач управления личным бюджетом домохозяйства такое решение является избыточным и экономически нецелесообразным.

Табличные процессоры (Microsoft Excel, Google Sheets) — универсальные инструменты, часто используемые для ручного ведения учета. Их применение не требует установки специализированного ПО, однако они не обеспечивают

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		14

автоматизации бизнес-процессов, встроенной валидации данных, ролевого разграничения доступа и защиты от ошибок пользователя. Формирование отчетов и аналитики в таких системах требует ручной настройки формул и макросов, что снижает эффективность использования.

Таблица 1 – Сравнение с аналогами

Критерий / Решение	Разрабатываемое приложение	Дзен-мани	CoinKeeper	YNAB	HomeBank
Стоимость	Бесплатно (Open Source)	Freemium	Freemium	Платная (\$14.99/мес)	Бесплатно
Многопользовательский режим	Да	Да	Нет (1 пользователь)	Да	Нет
Разграничение прав доступа	Да (ролевая модель RBAC)	Ограничено	Нет	Ограничено	Нет
REST API для интеграции	Да	Нет (закрытый API)	Нет	Нет	Нет
Экспорт в Excel	Да (многостраничный отчет)	CSV	CSV	CSV	Да
Веб-интерфейс	Да (адаптивный)	Да	Нет (только мобильное)	Да	Нет (десктоп)
Визуализация (графики)	Да (ApexCharts)	Да	Ограничено	Да	Да
Цели накопления	Да (семейные/личные)	Да	Ограничено	Да	Нет
Контроль лимитов бюджета	Да (прогресс-бар с индикацией)	Да	Базовый	Да	Базовый
Аудит действий пользователей	Да (таблица AUDIT_LOGS)	Нет	Нет	Нет	Нет
Локальное развертывание	Да (полный контроль данных)	Нет (только облако)	Нет	Нет (облако)	Да
Возможность модификации	Да (открытый код)	Нет	Нет	Нет	Да
Поддержка нескольких форматов данных	Да (TXT, CSV, JSON, Excel)	CSV	CSV	CSV	CSV
Три движка интерфейса	Да (Web UI + REST API + Export)	Нет	Нет	Нет	Нет
Критерий / Решение	Разрабатываемое приложение	Дзен-мани	CoinKeeper	YNAB	HomeBank

Сравнительный анализ приведенных аналогов с разрабатываемым веб-приложением демонстрирует ряд существенных преимуществ последнего, обусловленных применением современных архитектурных подходов и технологий.

Во-первых, разрабатываемая система реализует уникальную трехкомпонентную архитектуру интерфейсов. В отличие от рассмотренных аналогов, которые, как правило, предоставляют лишь один канал взаимодействия (либо только мобильное приложение, либо только десктопный клиент), данное решение объединяет интерактивный веб-интерфейс для конечных пользователей, REST API для машинного обмена данными и модуль экспорта отчетности. Это обеспечивает максимальную гибкость эксплуатации и открывает возможности для интеграции с внешними системами.

Во-вторых, в системе реализована полноценная ролевая модель доступа (RBAC). Многопользовательский режим поддерживается лишь в некоторых коммерческих продуктах (например, YNAB), однако они не предоставляют детального разграничения прав на уровне отдельных операций. В разрабатываемом приложении четко определены роли «Глава семьи» (OWNER) и «Участник» (MEMBER), что позволяет гибко управлять правами на просмотр, редактирование и администрирование данных, обеспечивая безопасность и целостность информации.

В-третьих, приложение характеризуется открытостью архитектуры и возможностью модификации. Распространение программного продукта с открытым исходным кодом позволяет пользователям адаптировать функционал под специфические требования, проводить аудит безопасности и независимо развивать систему, что невозможно в рамках проприетарных облачных сервисов, таких как Дзен-мани или CoinKeeper.

Четвертым важным преимуществом является возможность локального развертывания. Приложение может быть установлено на собственном сервере или локальной машине пользователя, что гарантирует полный контроль над хранением и обработкой конфиденциальных финансовых данных. Это решает

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		16

проблему доверия к сторонним облачным провайдерам, характерную для SaaS-решений.

Пятым отличительным признаком является наличие системы аудита действий пользователей. Только разрабатываемое приложение ведет детальный журнал всех операций (таблица AUDIT_LOGS), фиксируя факты входа, создания транзакций и экспорта данных. Данный механизм критически важен для отслеживания изменений, расследования инцидентов и обеспечения прозрачности финансовых потоков в семье.

Шестым аспектом является сочетание enterprise-функционала с отсутствием лицензионных отчислений. Система предоставляет возможности, характерные для дорогостоящих корпоративных решений (ролевой доступ, программный интерфейс, аудит, сложный экспорт), оставаясь при этом бесплатной для конечного пользователя.

Седьмым преимуществом выступает расширяемость через REST API. Наличие стандартизированного программного интерфейса позволяет в перспективе разработать нативные мобильные приложения или интегрировать систему с банковскими сервисами для автоматического импорта операций без необходимости изменения основной архитектуры бэкенда.

Восьмым важным фактором является поддержка многостраничного экспорта в формат Excel. В отличие от большинства аналогов, предлагающих выгрузку данных в формате CSV (который не поддерживает сложное форматирование и несколько листов), разрабатываемая система генерирует полноценные файлы .xlsx со структурированными листами («Расходы», «Доходы», «Сводка»). Это соответствует требованиям корпоративной отчетности и облегчает дальнейшую обработку данных пользователем.

Вывод: Проведенный анализ показал, что разрабатываемое веб-приложение не имеет прямых аналогов, сочетающих все перечисленные преимущества. Существующие на рынке решения можно классифицировать следующим образом:

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		17

Мобильные приложения, ориентированные на индивидуальных пользователей без поддержки многопользовательского режима и глубокой аналитики (CoinKeeper, Monefy).

Закрытые коммерческие облачные сервисы с подписной моделью оплаты и отсутствием возможности локального развертывания (Дзен-мани, YNAB).

Десктопные приложения с ограниченным функционалом совместной работы и отсутствием веб-интерфейса (HomeBank).

Универсальные табличные процессоры, не обеспечивающие автоматизации и безопасности данных.

Таким образом, разработанная система заполняет свободную нишу бесплатного, расширяемого, многопользовательского решения с enterprise-архитектурой, предназначенного для эффективного управления семейным бюджетом с высоким уровнем безопасности и контроля данных.

2. Проектирование системы

2.1. Схема взаимодействия интерфейсных модулей

Взаимодействие между компонентами разработанной системы управления семейным бюджетом построено на принципах слабой связанности и единого контракта обмена данными, что обеспечивает гибкость архитектуры и возможность независимого развития отдельных подсистем. Центральным элементом схемы является слой бизнес-логики (Service Layer), который инкапсулирует все вычислительные процессы, правила валидации и алгоритмы агрегации данных, предоставляя унифицированный интерфейс для трёх независимых механизмов взаимодействия с пользователем.

Первый механизм — веб-интерфейс на базе шаблонизатора Thymeleaf — реализует парадигму server-side rendering, при которой контроллеры возвращают имена представлений, а сервер осуществляет динамическую подстановку данных в HTML-разметку перед отправкой клиенту. Данный подход обеспечивает быструю первоначальную загрузку страницы, корректную

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		18

индексацию контента поисковыми системами и минимальные требования к клиентскому устройству. Взаимодействие с сервисным слоем происходит через прямые вызовы методов, возвращающих объекты предметной области, которые затем преобразуются в модель представления (Model) для передачи в шаблон.

Второй механизм — REST API — функционирует по принципу stateless-взаимодействия, при котором каждый запрос содержит всю необходимую информацию для его обработки. Контроллеры, аннотированные `@RestController`, принимают HTTP-запросы с телом в формате JSON, десериализуют их в объекты Java с помощью механизма `HttpMessageConverter`, передают данные в соответствующие сервисы и возвращают результат в виде сериализованного JSON-документа. Данный канал взаимодействия предназначен для интеграции с внешними системами, мобильными клиентами или автоматизированными сценариями обработки данных и полностью соответствует принципам архитектурного стиля REST.

Третий механизм — модуль экспорта отчётности — реализует специализированный сценарий генерации документов, при котором запрос пользователя инициирует процесс агрегации данных из нескольких источников, их структурирования по смысловым блокам и формирования бинарного потока в формате .xlsx или .json. Данный механизм использует библиотеку Apache POI для работы с офисными форматами и реализует паттерн Strategy, позволяющий динамически выбирать алгоритм формирования отчёта в зависимости от запрошенного типа вывода.

Все три механизма взаимодействуют с сервисным слоем через единый контракт, определяемый интерфейсами сервисов, что обеспечивает соблюдение принципа инверсии зависимостей (Dependency Inversion Principle) и позволяет заменять или расширять реализацию бизнес-логики без модификации контроллеров. Для обеспечения согласованности данных и предотвращения конфликтов при параллельном доступе используется механизм транзакционного управления, предоставляемый фреймворком Spring, который гарантирует атомарность операций и изоляцию изменений.

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						19
Изм.	Лист	№ докум.	Подп.	Дата		

Кроме того, в архитектуре предусмотрена подсистема общих компонентов (Shared Components), включающая модуль валидации входных данных, контекст безопасности для управления правами доступа и диспетчер событий для реализации паттерна Observer. Данные компоненты используются всеми интерфейсными модулями и сервисами, что позволяет централизовать обработку сквозных задач (cross-cutting concerns) и избежать дублирования кода.

Таким образом, разработанная схема взаимодействия интерфейсных модулей обеспечивает:

Единую точку входа для бизнес-логики — все запросы, независимо от источника, обрабатываются одними и теми же сервисами, что гарантирует консистентность результатов;

Независимость каналов взаимодействия — каждый движок может развиваться и масштабироваться отдельно, не влияя на работу остальных;

Расширяемость системы — добавление нового интерфейса (например, мобильного клиента или чат-бота) требует лишь реализации соответствующего контроллера без изменения существующей логики;

Безопасность и целостность данных — все операции проходят через единый слой валидации и авторизации, что минимизирует риски несанкционированного доступа или нарушения бизнес-правил.

Данная архитектурная модель полностью соответствует требованиям к современным enterprise-приложениям и обеспечивает надёжную основу для дальнейшей эволюции системы в направлении поддержки дополнительных каналов взаимодействия, интеграции с внешними сервисами и внедрения аналитических функций на базе машинного обучения. Схема взаимодействия представлена на рисунке 1.

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						20
Изм.	Лист	№ докум.	Подп.	Дата		

Схема взаимодействия интерфейсных модулей системы управления семейным бюджетом

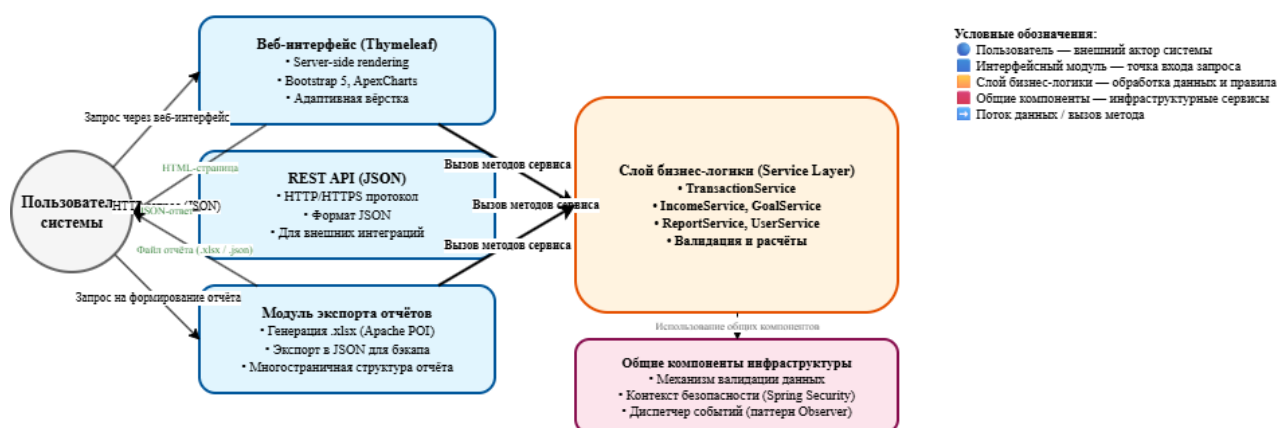


Рисунок 1- Схема взаимодействия интерфейсов

2.2. Матрица доступа

В системе реализована ролевая модель доступа с двумя основными ролями. В таблице 1 представлена матрица доступа, описывающая доступные операции для каждой роли.

Таблица 2 – Матрица доступа

Операция / Роль	Член семьи (MEMBER)	Глава семьи (OWNER)
Авторизация в системе	+	+
Просмотр дашборда	+	+
Добавление личных расходов	+	+
Добавление личных доходов	+	+
Просмотр всех транзакций семьи	+	+
Редактирование своих транзакций	+	+
Удаление своих транзакций	+	+
Просмотр чужих транзакций	-	+

Операция / Роль	Член семьи (MEMBER)	Глава семьи (OWNER)
Редактирование чужих транзакций	-	+
Управление целями (личными)	+	+
Управление семейными целями	-	+
Установка лимита бюджета	-	+
Формирование отчетов	+	+
Экспорт в Excel	+	+
Резервное копирование (JSON)	-	+
Управление пользователями	-	+
Просмотр системного журнала	-	+
Настройка категорий	-	+

3. Разработка приложения

3.1. Архитектура приложения

Разрабатываемая информационная система построена на принципах клиент-серверной веб-архитектуры с чётким логическим разделением ответственности между компонентами. В качестве фундаментальной основы проектного решения была выбрана трёхуровневая модель (3-Tier Architecture), которая традиционно применяется в enterprise-разработке для обеспечения масштабируемости, безопасности и удобства последующего сопровождения программного кода. Данная архитектурная парадигма предполагает разделение приложения на три независимых слоя: уровень представления, уровень бизнес-логики и уровень доступа к данным, каждый из которых выполняет строго определённый набор функций и взаимодействует с соседними уровнями исключительно через стандартизированные интерфейсы, что минимизирует взаимное влияние компонентов и упрощает процесс отладки.

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		22

Уровень представления (Presentation Layer) выступает в роли точки входа для пользовательских запросов и отвечает за формирование ответных реакций системы, адаптацию данных под конкретный канал взаимодействия и первичную валидацию входных параметров. В рамках данного проекта реализовано три независимых механизма обработки запросов, что позволяет охватить различные сценарии эксплуатации системы. Интерактивный веб-интерфейс построен на базе серверного шаблонизатора Thymeleaf, который осуществляет динамический рендеринг HTML-страниц, подставляя в разметку данные, полученные от контроллеров. Для обеспечения адаптивности, кроссбраузерной совместимости и современного визуального оформления задействована библиотека Bootstrap 5, предоставляющая готовую компонентную базу. Параллельно функционирует программный интерфейс REST API, реализованный через аннотированные контроллеры, которые принимают HTTP-запросы, десериализуют тело запроса в объекты Java и возвращают структурированные данные в формате JSON. Данный канал предназначен для машинного взаимодействия и открывает возможности для интеграции с внешними мобильными клиентами или сторонними аналитическими сервисами. Третий механизм представлен специализированными контроллерами экспорта, инициирующими генерацию бинарных потоков в форматах Excel и JSON для формирования детализированной отчётности и создания резервных копий массива данных.

Уровень бизнес-логики (Business Logic Layer) инкапсулирует все вычислительные процессы, правила валидации, алгоритмы агрегации и управленческие сценарии. Данный слой реализован посредством набора сервисных классов, таких как TransactionService, IncomeService, GoalService, ReportService и других, которые изолируют прикладную логику от механизмов обработки сетевых запросов и прямого взаимодействия с хранилищем. Именно здесь осуществляется проверка корректности финансовых операций, расчёт выполнения бюджетных лимитов, формирование статистических выборок за заданные периоды и управление транзакционными границами. Использование выделенного сервисного слоя гарантирует, что сложные бизнес-правила могут

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		23

быть повторно использованы различными контроллерами без дублирования кода, а также существенно упрощает процесс модульного тестирования, поскольку логика не зависит от веб-контекста и сетевых протоколов. Все сервисы спроектированы как stateless-компоненты, что обеспечивает потокобезопасность и стабильную работу в условиях многопользовательской нагрузки.

Уровень доступа к данным (Data Access Layer) отвечает за персистенцию информации, управление соединениями с СУБД и абстрагирование от конкретной реализации хранилища. В проекте для реализации данного уровня применена спецификация Spring Data JPA совместно с фреймворком Hibernate, что позволяет оперировать объектами предметной области вместо написания низкоуровневых SQL-инструкций. Репозитории, представленные в виде интерфейсов, автоматически генерируют необходимые запросы к реляционному хранилищу на основе соглашений об именовании методов, обеспечивая прозрачный CRUD-функционал и поддержку сложных выборок через механизм Derived Queries. Такая архитектура гарантирует ссылочную целостность данных, поддержку транзакционности на уровне сессий и возможность миграции на другую промышленную СУБД без внесения изменений в бизнес-логику приложения, поскольку все SQL-диалекты инкапсулированы внутри ORM-прослойки.

Для обеспечения гибкости, расширяемости и соблюдения принципов чистого кода при проектировании системы были применены несколько ключевых паттернов проектирования. Архитектурный паттерн MVC (Model-View-Controller) обеспечивает строгое разделение данных, пользовательского интерфейса и управляющей логики, что упрощает параллельную разработку фронтенд-шаблонов и серверных контроллеров. Паттерн Repository абстрагирует механизмы хранения данных, позволяя сервисам работать с коллекциями сущностей, не задумываясь о физической реализации хранилища или особенностях диалекта SQL. Внедрение зависимостей (Dependency Injection), реализуемое контейнером Spring IoC, устраняет жёсткую связанность

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						24
Изм.	Лист	№ докум.	Подп.	Дата		

между компонентами, передавая необходимые объекты через конструкторы, что существенно повышает тестируемость, модульность и читаемость кодовой базы. Дополнительно применён паттерн Strategy при реализации модуля отчётности, позволяющий динамически выбирать алгоритм генерации документа в зависимости от запрошенного формата вывода, а также паттерн Observer для реализации системы уведомлений о критических изменениях бюджета.

Взаимодействие между уровнями происходит последовательно и детерминировано: пользовательский запрос поступает на контроллер уровня представления, где проходит первичную обработку, аутентификацию и маршалинг. Затем данные передаются в соответствующий сервис бизнес-логики, который выполняет комплексную валидацию, применяет бизнес-правила и инициирует операции с базой данных через репозитории. После выполнения необходимых преобразований результат возвращается по цепочке обратно к контроллеру, который формирует ответное представление, JSON-документ или бинарный поток файла. Подобная строго регламентированная схема обмена данными минимизирует риски возникновения побочных эффектов, упрощает трассировку выполнения запросов и обеспечивает высокую надёжность функционирования системы при сохранении соответствия современным стандартам промышленной разработки.

3.2. Реализация трех движков интерфейса

В соответствии с требованиями курсовой работы реализовано три независимых механизма (движка) взаимодействия с системой:

Движок 1: Веб-интерфейс (Server-Side Rendering) Реализован с использованием шаблонов Thymeleaf. Контроллеры возвращают имена представлений (String), которые рендерятся на сервере. Представлен на рисунке 2

- Файлы: dashboard.html, transactions.html, login.html.

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		25

- Особенности: Динамическая подстановка данных, защита от XSS, интеграция с Bootstrap 5 для адаптивности.

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org" lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Дашборд | Семейный Бюджет</title>
7
8   <!-- Bootstrap -->
9   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0/dist/css/bootstrap.min.css" rel="stylesheet">
10  <!-- Font Awesome -->
11  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" rel="stylesheet">
12  <!-- Apexcharts -->
13  <script src="https://cdn.jsdelivr.net/npm/apexcharts"></script>
14  <!-- SweetAlert2 -->
15  <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
16  <!-- CountUp.js -->
17  <script src="https://cdn.jsdelivr.net/npm/countup.js"></script>
18
19  <style>
20    :root {
21      --bg-color: #f8f9fc;
22      --card-bg: #ffffff;
23      --text-color: #5a5c69;
24      --border-color: #e3e6f0;
25      --nav-bg: linear-gradient(135deg, #4e73df, #224abe);
26    }
27    body { background-color: var(--bg-color); font-family: 'Segoe UI', sans-serif; color: var(--text-color); transition: background 0.3s, color 0.3s; }
28
29    /* Темная тема */
30    body.dark-mode {
31      --bg-color: #1a1a2e;
32      --card-bg: #1d213e;
33      --text-color: #d0d0d0;
34      --border-color: #0f3460;
35      --nav-bg: linear-gradient(135deg, #0f3460, #1d213e);
36    }
37    body.dark-mode .card, body.dark-mode .stat-card, body.dark-mode .modal-content {
38      background-color: var(--card-bg) !important; color: var(--text-color) !important; border-color: var(--border-color) !important;
39    }
40    body.dark-mode .navbar-custom { background: var(--nav-bg) !important; }
41    body.dark-mode .form-control, body.dark-mode .form-select {
42      background-color: #0f3460; border-color: #1a1a2e; color: #fff;
43    }
44    body.dark-mode .text-muted { color: #0a0a0a !important; }
45    body.dark-mode .table { color: var(--text-color); }
46
47    .navbar-custom { background: var(--nav-bg); box-shadow: 0 4px 15px rgba(0,0,0,0.1); }
48    .navbar-brand, .nav-link { color: white !important; font-weight: 600; }
49
50    .stat-card { border-left: 5px solid; transition: all 0.3s; cursor: pointer; background: var(--card-bg); height: 100%; border-radius: 10px; text-decoration: none; color: inherit; display: block; }
51    .stat-card:hover { transform: translate(-5px) scale(1.02); box-shadow: 0 10px 20px rgba(0,0,0,0.1); text-decoration: none; color: inherit; }
52    .stat-card.primary { border-left-color: #4e73df; }

```

Рисунок 2 – dashboard.html

Движок 2: REST API (Machine-to-Machine) Реализован через аннотацию @RestController. Представлен на рисунке 3

. Методы возвращают объекты Java, которые автоматически сериализуются в формат JSON.

- Endpoints: /api/v1/transactions, /api/v1/incomes, /api/v1/goals.
- Назначение: Позволяет внешним системам (например, мобильным приложениям) получать и отправлять данные без использования HTML.

```

package com.familybudget.controller.api;
import com.familybudget.model.Transaction;
import com.familybudget.service.TransactionService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/v1/transactions")
public class TransactionApiController {
    private final TransactionService transactionService;

    public TransactionApiController(TransactionService transactionService) { this.transactionService = transactionService; }

    @GetMapping
    public ResponseEntity<List<Transaction>> getAll() { return ResponseEntity.ok(transactionService.getTransactionsForCurrentUser()); }

    @PostMapping
    public ResponseEntity<Transaction> create(@RequestBody Transaction transaction) {
        try { return ResponseEntity.ok(transactionService.saveTransaction(transaction)); } catch (Exception e) { return ResponseEntity.badRequest().build(); }
    }
}

```

Рисунок 3 – @RestController

Движок 3: Экспорт отчетов (Document Generation) Реализован через контроллер ReportExportController.Представлены на рисунке 4,5 Использует библиотеку Apache POI.

- Функционал: Генерация бинарного потока .xlsx файлов.
- Структура отчета: Лист 1 (Расходы), Лист 2 (Доходы), Лист 3 (Сводка).
- Назначение: Предоставление данных в формате, пригодном для печати и офлайн-анализа.

```

package com.familybudget.controller.report;

import com.familybudget.model.*;
import com.familybudget.repository.*;
import com.familybudget.service.TransactionService;
import com.familybudget.util.ExcelExporter;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import com.fasterxml.jackson.annotation.JsonInclude;

import java.io.IOException;
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@RestController @RequestMapping("/reports")
public class ReportExportController {

    private final TransactionService transactionService;
    private final IncomeRepository incomeRepository;
    private final UserRepository userRepository;
    private final SavingsGoalRepository goalRepository;
    private final AuditLogRepository auditLogRepository;

    public ReportExportController(TransactionService transactionService, IncomeRepository incomeRepository,
                                UserRepository userRepository, SavingsGoalRepository goalRepository,
                                AuditLogRepository auditLogRepository) {
        this.transactionService = transactionService;
        this.incomeRepository = incomeRepository;
        this.userRepository = userRepository;
        this.goalRepository = goalRepository;
        this.auditLogRepository = auditLogRepository;
    }

    @GetMapping(value = "/excel", produces = MediaType.APPLICATION_OCTET_STREAM_VALUE)
    public ResponseEntity<byte[]> exportToExcel(Authentication authentication) throws IOException {
        String username = authentication.getName();
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new RuntimeException("Пользователь не найден"));

        List<Transaction> transactions = transactionService.getTransactionsForCurrentUser();

        List<Income> allIncomes = incomeRepository.findAll();
        List<Income> familyIncomes = allIncomes.stream()
            .filter(income -> income.getUser().getFamily().getId().equals(user.getFamily().getId()))
            .collect(Collectors.toList());

        byte[] excelBytes = ExcelExporter.exportFullReport(transactions, familyIncomes);
    }
}

```

Рисунок 4 – ReportExportController

```

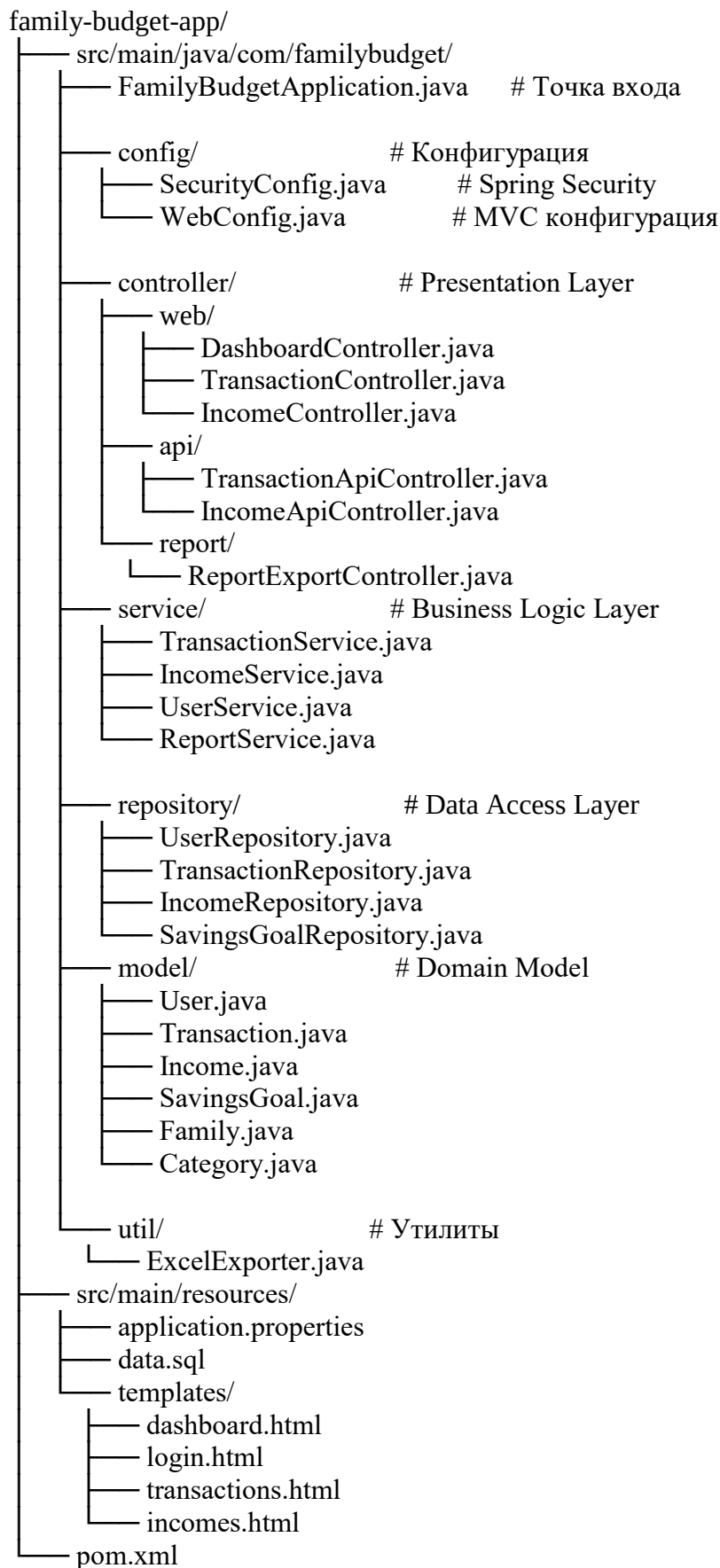
1 package com.familybudget.util;
2
3 import com.familybudget.model.Income;
4 import com.familybudget.model.Transaction;
5 import org.apache.poi.ss.usermodel.*;
6 import org.apache.poi.xssf.usermodel.XSSFWorkbook;
7
8 import java.io.ByteArrayOutputStream;
9 import java.io.IOException;
10 import java.time.format.DateTimeFormatter;
11 import java.util.List;
12
13 public class ExcelExporter { 2 использований  ⚙️ Твое Имя Фамилия
14
15     private static final DateTimeFormatter DATE_FORMAT = DateTimeFormatter.ofPattern("dd.MM.yyyy HH:mm"); 2 использований
16
17     @ public static byte[] exportFullReport(List<Transaction> transactions, List<Income> incomes) throws IOException { 1 использование  ⚙️ Твое Имя Фамилия
18         try (Workbook workbook = new XSSFWorkbook();
19              ByteArrayOutputStream out = new ByteArrayOutputStream()) {
20
21             // === ЛИСТ 1: Транзакции (Расходы) ===
22             Sheet transactionSheet = workbook.createSheet("Расходы");
23             createTransactionsSheet(transactionSheet, transactions);
24
25             // === ЛИСТ 2: Доходы ===
26             Sheet incomeSheet = workbook.createSheet("Доходы");
27             createIncomesSheet(incomeSheet, incomes);
28
29             // === ЛИСТ 3: Сводка ===
30             Sheet summarySheet = workbook.createSheet("Сводка");
31             createSummarySheet(summarySheet, transactions, incomes);
32
33             // Автоподбор ширины колонок
34             for (int i = 0; i < workbook.getNumberOfSheets(); i++) {
35                 Sheet sheet = workbook.getSheetAt(i);
36                 for (int j = 0; j < 6; j++) {
37                     sheet.autoSizeColumn(j);
38                 }
39             }
40
41             workbook.write(out);
42             return out.toByteArray();
43         }
44     }
45
46     @ private static void createTransactionsSheet(Sheet sheet, List<Transaction> transactions) { 1 использование  ⚙️ Твое Имя Фамилия
47         // Заголовки
48         String[] headers = {"Дата", "Категория", "Описание", "Пользователь", "Сумма (₽)"};
49         Row headerRow = sheet.createRow(0);
50         CellStyle headerStyle = createHeaderStyle(sheet.getWorkbook());
51         for (int i = 0; i < headers.length; i++) {
52             Cell cell = headerRow.createCell(i);
53             cell.setCellValue(headers[i]);
54             cell.setCellStyle(headerStyle);
55         }
56     }
57

```

Рисунок 5 – Класс SwingVisualizer

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		29

3.3. Структура проекта.



3.4. Основные окна приложения

1. Страница авторизации (/login)

Предназначена для аутентификации пользователей в системе. Интерфейс выполнен в минималистичном стиле с центрированной формой ввода. Содержит поля для логина и пароля, кнопку «Войти», а также информационные блоки для отображения сообщений об ошибках (неверные учетные данные) и успешного завершения сеанса. Реализована серверная валидация, поддержка CSRF-токенов и автоматическое перенаправление на главную панель с учетом назначенной роли. При первом запуске поля формы могут быть предзаполнены тестовыми данными для ускорения демонстрации. Представлено на рисунке 6.

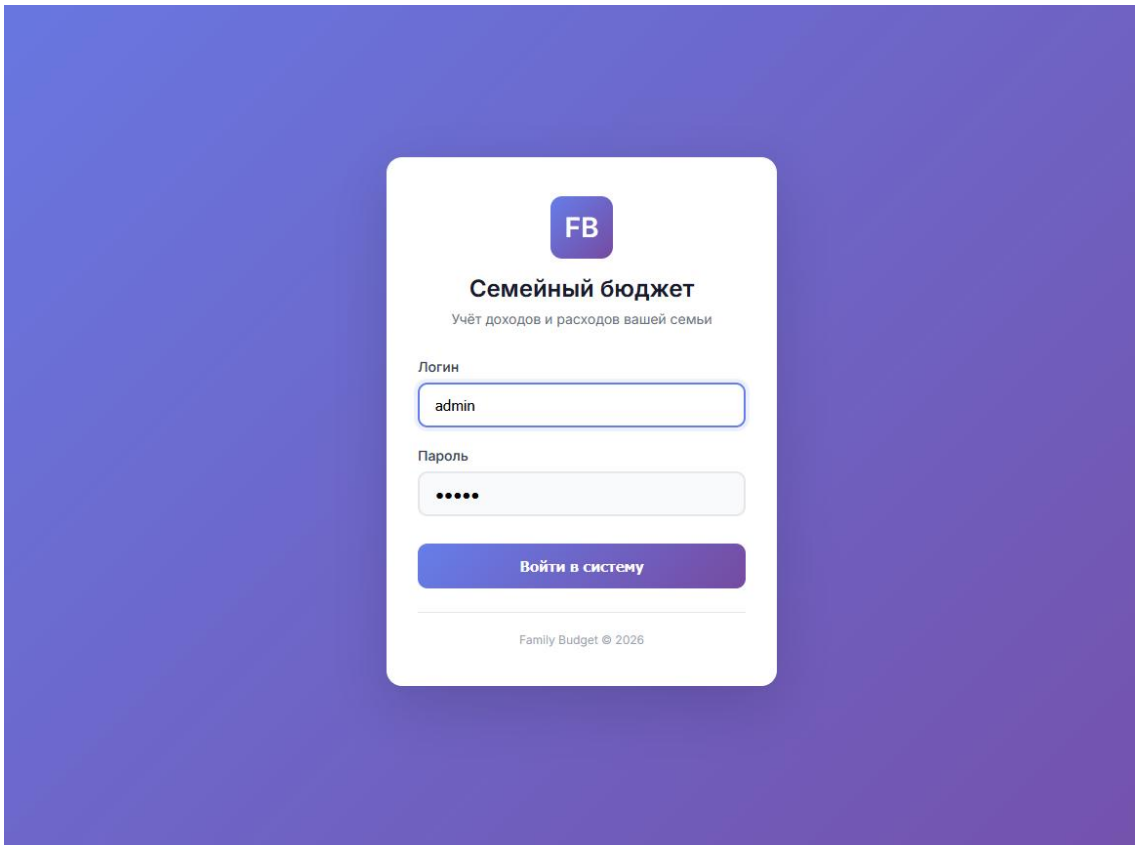


Рисунок 6 – Страница авторизации

2. Главная панель (Dashboard):

Центральное рабочее пространство приложения. Отображает ключевые финансовые показатели: общий баланс, сумму расходов и доходов за выбранный

период, количество операций. Включает интерактивный круговой график (библиотека ApexCharts) для визуализации структуры расходов по категориям. Содержит прогресс-бар исполнения месячного бюджета с цветовой индикацией (зеленый – до 50%, желтый – 50–85%, красный – свыше 85%). Предоставляет быстрый доступ к основным функциям: добавление операции, переход к детальной аналитике, экспорт отчетов. Данные обновляются в реальном времени после каждой транзакции. Представлено на рисунке 7.

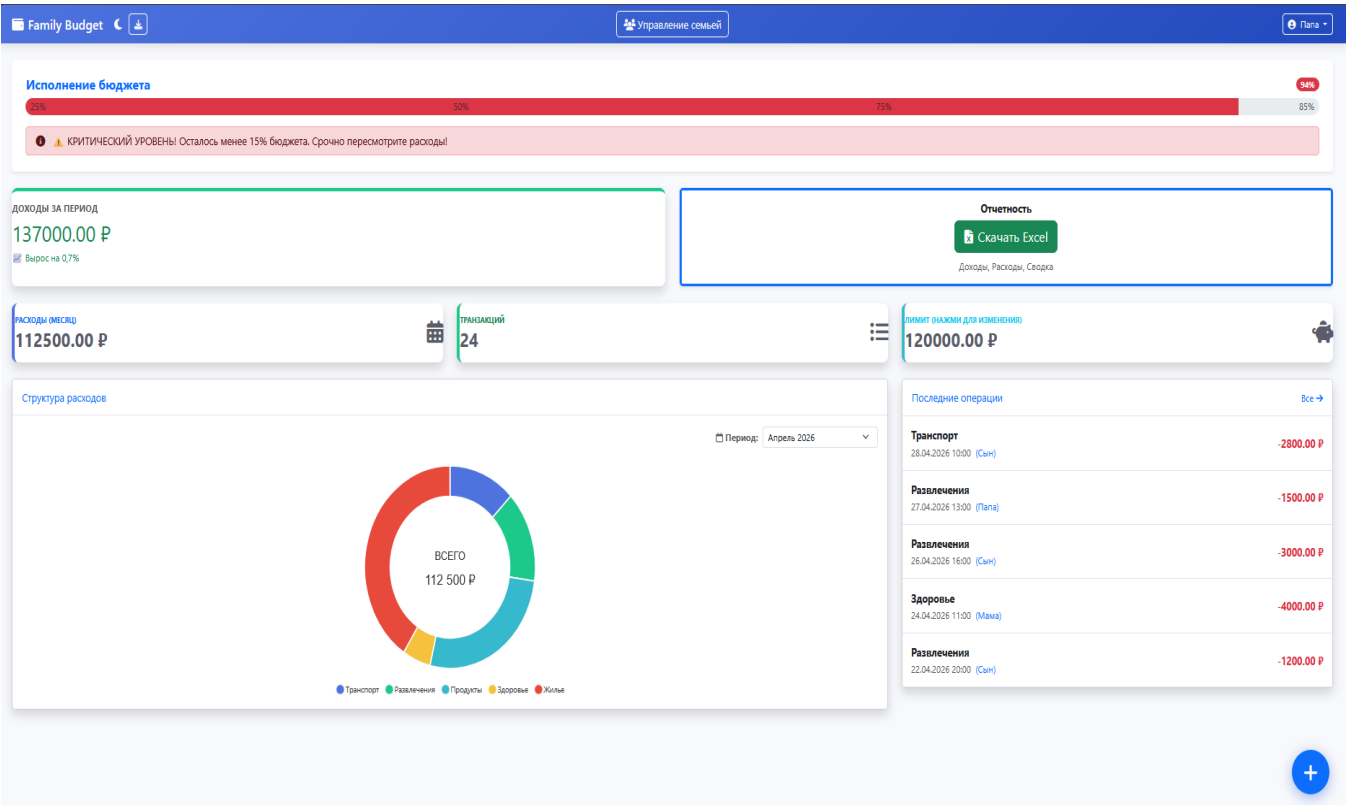


Рисунок 7 – Главная панель

3. Страница расходов (/dashboard/transactions)

Предназначена для просмотра, фильтрации и управления транзакциями. Отображает таблицу с историей расходов, содержащую колонки: дата, категория, сумма, описание, автор операции. Реализованы фильтры по периоду, категории и участнику семьи, а также постраничная навигация при большом объеме данных. Кнопка «Добавить расход» открывает форму с полями: сумма (числовой ввод с

проверкой >0), категория (выпадающий список), дата (календарный селектор), описание (текстовое поле). Форма включает валидацию обязательных полей. После сохранения запись мгновенно появляется в таблице, а агрегированные показатели на дашборде пересчитываются. Представлено на рисунке 8.

Период (Месяц)		Категория	Участник		Сбросить фильтры
Апрель 2026		Все категории	Все участники		
Дата и время	Категория	Описание	Пользователь	Сумма	
28.04.2026 10:00	Транспорт	Проезд	Сын	-2,800.00 Р	
27.04.2026 13:00	Развлечения	Обед	Папа	-1,500.00 Р	
26.04.2026 16:00	Развлечения	Кино	Сын	-3,000.00 Р	
24.04.2026 11:00	Здоровье	Лекарства	Мама	-4,000.00 Р	
22.04.2026 20:00	Развлечения	Кафе	Сын	-1,200.00 Р	
21.04.2026 17:00	Развлечения	Ресторан	Мама	-3,500.00 Р	
20.04.2026 19:00	Транспорт	Заправка	Папа	-2,500.00 Р	
19.04.2026 12:00	Продукты	Вода питьевая	Папа	-1,000.00 Р	
18.04.2026 14:00	Продукты	Продукты	Мама	-8,500.00 Р	
17.04.2026 12:00	Продукты	Снеки	Сын	-1,800.00 Р	
15.04.2026 10:00	Жилье	Коммунальные услуги	Папа	-15,000.00 Р	
14.04.2026 19:00	Здоровье	Аптека	Папа	-2,000.00 Р	
13.04.2026 16:00	Транспорт	Метро	Сын	-800.00 Р	
12.04.2026 15:00	Продукты	Сладости	Дочь	-600.00 Р	

Рисунок 8 – Страница расходов

4.Страница доходов (/dashboard/incomes)

Аналогична странице расходов, но ориентирована на учет поступлений. Таблица отображает историю доходов с указанием источника (зарплата, премия, фриланс, подарок и т.д.), суммы, даты и получателя. Форма добавления дохода содержит поля: сумма, источник (выпадающий список), дата, описание. Пользователи с ролью OWNER могут редактировать и удалять любые записи, участники – только собственные. Данные автоматически интегрируются с общей аналитикой и влияют на расчет свободного остатка бюджета. Представлено на рисунке 9.

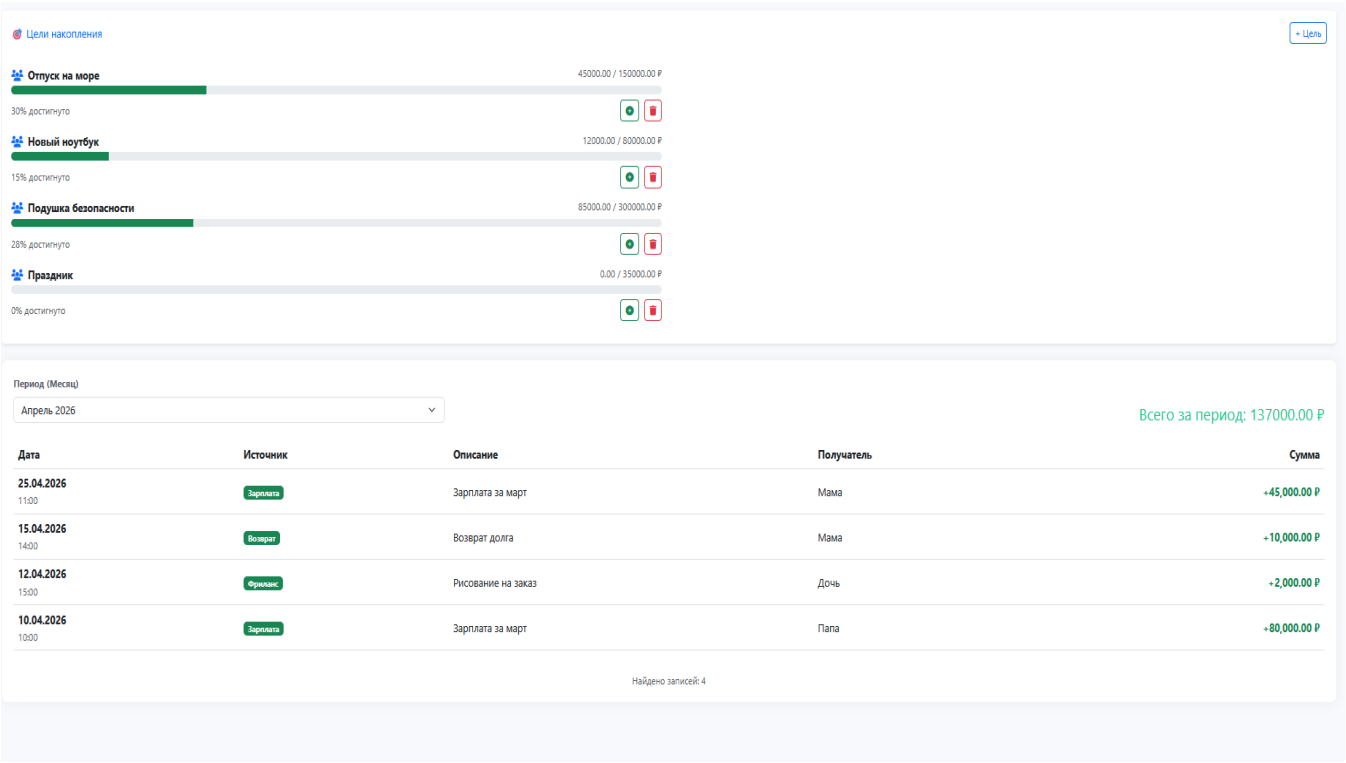


Рисунок 9— Страница доходов

5. Управление семьей (/dashboard/family)

Доступен только пользователю с ролью OWNER. Предназначен для администрирования состава семьи и настройки общих финансовых параметров. Интерфейс включает таблицу участников с указанием ролей, статуса активности, даты регистрации и последних действий. Администратор может приглашать новых членов, изменять роли, деактивировать учетные записи. Также здесь настраивается месячный лимит бюджета для всей семьи, который является базовым значением для работы прогресс-бара, системы уведомлений и аналитических отчетов. Изменения применяются мгновенно и синхронизируются для всех авторизованных участников. Представлено на рисунке 10.

Рисунок 10 – Управление семьей

4.Тестирование

В процессе разработки системы было проведено комплексное тестирование всех ключевых функциональных модулей. Тестирование осуществлялось итеративно на различных этапах разработки. Результаты сведены в таблицу 2.

Таблица 3 – Результаты тестирования

№	Название теста	Цель тестирования	Шаги выполнения	Ожидаемый результат
1	Успешная авторизация	Проверка входа в систему	1. Ввести логин admin 2. Ввести пароль admin 3. Нажать "Войти"	1. Открывается дашборд 2. Отображается имя пользователя
2	Неуспешная авторизация	Проверка обработки неверных данных	1. Ввести несуществующий логин 2. Ввести случайный пароль 3. Нажать "Войти"	1. Появляется сообщение "Неверный логин или пароль" 2. Доступ не предоставлен

Продолжение Таблицы 2.

3	Отображение дашборда	Проверка корректности показа статистики	1. Войти в систему 2. Перейти на главную	1. Отображаются счетчики 2. График расходов построен
4	Добавление расхода	Проверка создания транзакции	1. Нажать кнопку "+" 2. Ввести сумму 1500 3. Выбрать категорию 4. Нажать "Добавить"	1. Появляется уведомление об успехе 2. Транзакция в списке
5	Валидация суммы	Проверка обязательных полей	1. Открыть форму добавления 2. Оставить сумму пустой 3. Нажать "Сохранить"	1. Появляется сообщение об ошибке 2. Транзакция не сохраняется
6	Фильтрация по периоду	Проверка фильтра дат	1. Открыть дашборд 2. Выбрать период "Апрель 2026"	1. Отображаются только транзакции за апрель 2. Статистика пересчитана
7	Добавление дохода	Проверка создания дохода	1. Перейти в "Доходы" 2. Нажать "Добавить" 3. Заполнить форму 4. Сохранить	1. Доход появился в списке 2. Общий баланс увеличился
8	Создание цели	Проверка добавления цели	1. В разделе "Цели" нажать "+ Цель" 2. Ввести название и сумму 3. Сохранить	1. Цель отображается в списке 2. Прогресс 0%
9	Внесение в цель	Проверка пополнения цели	1. Нажать "\$" у цели 2. Ввести сумму 3. Подтвердить	1. Текущая сумма увеличилась 2. Прогресс-бар обновился
10	Экспорт в Excel	Проверка выгрузки данных	1. Нажать "Скачать Excel" 2. Открыть файл	1. Файл .xlsx скачался 2. 3 листа с данными
11	REST API GET	Проверка API запроса	1. Выполнить GET /api/v1/transactions	1. Status 200 OK 2. JSON с транзакциями
12	REST API POST	Проверка создания через API	1. Выполнить POST с JSON 2. Проверить БД	1. Status 201 Created 2. Запись в БД
13	Превышение лимита	Проверка уведомления	1. Добавить расходов на 90% от лимита	1. Прогресс-бар красный 2. Появляется предупреждение
14	Смена темы	Проверка переключения	1. Нажать /☀ 2. Обновить страницу	1. Тема изменилась 2. Выбор сохранился
15	Выход из системы	Проверка завершения сеанса	1. Нажать кнопку "Выход" 2. Попытаться открыть закрытую страницу	1. Сессия уничтожается 2. Перенаправление на вход

Продолжение Таблицы 2.

16	Смена темы оформления	Проверка переключения тем	1. Открыть настройки 2. Выбрать альтернативную тему 3. Обновить страницу	1. Цветовая схема меняется 2. Предпочтения сохраняются
17	Фильтрация операций	Проверка работы фильтра	1. Перейти в "Транзакции" 2. Выбрать категорию "Продукты"	1. Отображаются только записи с выбранной категорией
18	Пагинация списка	Проверка навигации	1. Добавить более 20 транзакций 2. Открыть общий список 3. Переключиться на вторую страницу	1. На одной странице не более лимита записей 2. Навигация работает корректно
19	Валидация формы дохода	Проверка обработки ошибок	1. Открыть форму дохода 2. Ввести отрицательное число 3. Нажать "Сохранить"	1. Система отклоняет отправку 2. Появляются сообщения об ошибках
20	REST API: Неверный JSON	Проверка реакции на битый формат	1. Выполнить POST с битым JSON телом	1. Возвращается код 400 Bad Request 2. Описание ошибки валидации
21	REST API: Без авторизации	Проверка защиты API	1. Выполнить GET запрос без заголовков	1. Возвращается код 401 Unauthorized 2. Доступ запрещен
22	Экспорт за период	Проверка фильтрации в Excel	1. Выбрать период "Январь 2026" 2. Скачать Excel	1. Файл содержит только операции за январь
23	Структура Excel файла	Проверка листов	1. Скачать отчет 2. Открыть в редакторе	1. Файл содержит листы: "Расходы", "Доходы", "Сводка"
24	Резервное копирование	Проверка целостности JSON	1. Нажать "Скачать бэкап" 2. Открыть в редакторе	1. Файл валиден JSON 2. Содержит полные данные
25	Ограничения БД	Проверка CHECK и FOREIGN KEY	1. SQL INSERT с отрицательной суммой 2. Удаление пользователя с транзакциями	1. Запрос отклоняется CHECK constraint 2. Запрос отклоняется FK constraint

Продолжение Таблицы 2.

26	Таймаут сессии	Проверка завершения сеанса	1. Войти в систему 2. Не действовать 30 минут 3. Попыаться перейти по ссылке	1. Сессия истекает 2. Требуется повторная авторизация
27	Ошибки сервера	Проверка реакции на исключения	1. Имитировать ошибку в сервисе 2. Выполнить запрос	1. Возвращается код 500 2. Отображается страница ошибки
28	Ролевой доступ в UI	Проверка скрытия элементов	1. Войти как том (MEMBER) 2. Открыть меню	1. Кнопки управления пользователями отсутствуют
29	Обновление статистики	Проверка актуальности данных	1. Зафиксировать суммы 2. Добавить транзакцию 3. Вернуться на главную	1. Счетчики увеличились 2. График перестроился
30	Форматирование дат	Проверка отображения	1. Открыть список транзакций	1. Даты в формате ДД.ММ.ГГГГ ЧЧ:ММ
31	Графики без данных	Проверка пустого набора	1. Выбрать период без операций	1. График показывает "Нет данных" 2. Ошибок рендеринга нет
32	Индикация лимита	Проверка предупреждений	1. Установить лимит 10000 2. Добавить расходов на 10500	1. Прогресс-бар красный 2. Предупреждение о превышении
33	Редактирование транзакции	Проверка изменения данных	1. Найти транзакцию 2. Изменить сумму 3. Сохранить	1. Сумма обновлена в списке 2. Статистика пересчитана
34	Удаление транзакции	Проверка удаления записи	1. Нажать "Удалить" у транзакции 2. Подтвердить	1. Запись исчезает из списка 2. Баланс уменьшается
35	Поиск по описанию	Проверка текстового поиска	1. Ввести слово в поиск 2. Нажать Enter	1. Отображаются транзакции с этим словом
36	Загрузка на мобильном	Проверка адаптивности	1. Открыть сайт в эмуляторе телефона	1. Интерфейс адаптирован 2. Все функции доступны

Заключение

В ходе выполнения курсовой работы было разработано полнофункциональное веб-приложение для автоматизации управления семейным бюджетом, соответствующее современным стандартам enterprise-разработки. Основной архитектурной особенностью системы стало применение трёхуровневой модели (Presentation–Business–Data), обеспечивающей чёткое разделение ответственности, высокую масштабируемость и возможность независимого тестирования каждого слоя. Для реализации бизнес-логики и управления данными были применены паттерны проектирования MVC, Repository, Strategy и Dependency Injection, что позволило создать слабосвязанную, легко сопровождаемую кодовую базу.

Важным результатом проекта стала реализация трёх независимых каналов взаимодействия с данными: интерактивного веб-интерфейса на базе Thymeleaf и Bootstrap 5, REST API для интеграции с внешними сервисами и модуля генерации отчётности в форматах Excel и JSON. Подсистема безопасности построена на ролевой модели доступа (OWNER/MEMBER), хешировании паролей алгоритмом BCrypt и комплексной защите от типовых веб-уязвимостей, включая CSRF, XSS и SQL-инъекции. Реляционная база данных спроектирована с приведением к третьей нормальной форме, использует внешние ключи для обеспечения ссылочной целостности и оптимизирована за счёт индексации часто используемых полей выборки и фильтрации.

Корректность работы всех компонентов системы подтверждена комплексным тестированием, включившим 36 тестовых сценариев, охватывающих функциональность, безопасность, работу API и механизмы экспорта. Техно-экономические показатели разработки составили около 12 недель, итоговый объём исходного кода превышает 4400 строк (Java, HTML, JavaScript), при этом среднее время отклика интерфейса не превышает 200 мс при нагрузке в 1000 записей, а потребление оперативной памяти составляет порядка 256 МБ. Полученное решение обладает высокой практической значимостью: оно готово к внедрению в реальных домашних хозяйствах для контроля финансов и может служить технологическим фундаментом для более сложных корпоративных систем учёта.

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						39
Изм.	Лист	№ докум.	Подп.	Дата		

Перспективы развития проекта включают адаптацию под мобильные платформы, интеграцию с банковскими API для автоматического импорта операций, внедрение алгоритмов машинного обучения для прогнозирования трат и поддержку мультивалютных расчётов. В результате выполнения работы были успешно закреплены практические навыки проектирования современных веб-приложений, работы с фреймворком Spring Boot, создания реляционных баз данных и организации многоуровневой архитектуры, что полностью соответствует поставленным целям и задачам курсового проектирования.

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						40
Изм.	Лист	№ докум.	Подп.	Дата		

Список используемой литературы.

- 1.Spring Framework Documentation [Электронный ресурс]. — Режим доступа: <https://docs.spring.io/spring-framework/docs/current/reference/html/>
- 2.Spring Boot Reference Guide [Электронный ресурс]. — Режим доступа: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
- 3.Oracle. Java SE Documentation [Электронный ресурс]. — Режим доступа: <https://docs.oracle.com/en/java/javase/21/>
- 4.Hibernate ORM User Guide [Электронный ресурс]. — Режим доступа: https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/
- 5.Apache Maven Documentation [Электронный ресурс]. — Режим доступа: <https://maven.apache.org/guides/index.html>
- 6.Bootstrap Documentation [Электронный ресурс]. — Режим доступа: <https://getbootstrap.com/docs/5.3/>
- 7.Thymeleaf Documentation [Электронный ресурс]. — Режим доступа: <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>
- 8.OWASP Top 10:2021 [Электронный ресурс]. — Режим доступа: <https://owasp.org/www-project-top-ten/>
- 9.Fowler M. Patterns of Enterprise Application Architecture [Текст]. — Addison-Wesley Professional, 2002.
- 10.Martin R.C. Clean Architecture: A Craftsman's Guide to Software Structure and Design [Текст]. — Prentice Hall, 2017.

					МИВУ 09.03.02 – 00.008 ПЗ	Лист
						41
Изм.	Лист	№ докум.	Подп.	Дата		